实验 3　文件系统实验

1. 实验目的

通过一个简单文件系统的设计，理解文件系统的内部结构、基本功能及实现。

2. 实验内容

1) 分析 EXT2 文件系统的结构；
2) 基于 Ext2 的思想和算法，设计一个类 Ext2 的多级文件系统，实现 Ext2 文件系统的一个功能子集；
3) 用现有操作系统上的文件来代替硬盘进行硬件模拟。

3. 实验思想

在分析 Linux 的文件系统的基础上，基于 Ext2 的思想和算法，设计一个类 Ext2 的虚拟多级文件系统，实现 Ext2 文件系统的一个功能子集。并且用现有操作系统上的文件来代替硬盘进行硬件模拟。设计文件系统应该考虑的几个层次：①介质的物理结构；②物理操作——设备驱动程序完成；③文件系统的组织结构（逻辑组织结构）；④对组织结构其上的操作；⑤为用户使用文件系统提供的接口。本实验只涉及后三个层次的内容。

4. 实验准备知识

1) Linux 文件系统简介

在 Linux 系统中，“一切皆文件”的理念深入人心，而 VFS（Virtual File System）正是这一理念的重要体现。VFS 是 Linux 内核中一个非常关键的抽象层，它的核心使命是为所有类型的文件系统提供一个统一的访问接口，为用户和应用程序提供一致的操作方式。新的文件系统只要支持并实现这些接口，并注册到 Linux 内核中，即可安装和使用。它将不同类型的文件系统，如基于磁盘的文件系统（如 ext4、XFS 等）、网络文件系统（如 NFS、CIFS 等）、虚拟文件系统（如 /proc、/sys 等），都统一抽象成可以通过文件操作接口（如 open、close、read、write 等）访问的对象。这使得应用程序可以使用统一的方式来操作不同类型的文件系统，大大提高了系统的通用性和可扩展性。

(1) VFS 工作原理

VFS 处于用户空间和底层具体文件系统之间，用户空间的应用程序通过系统调用接口

（System Call Interface）与内核进行交互，而 VFS 则负责处理这些系统调用，并将其转发到底层的具体文件系统。同时，它又把底层文件系统返回的结果，以统一的格式呈现给用户和应用程序。这样一来，用户和应用程序就完全不用关心底层文件系统的具体实现细节，只需要和 VFS 进行交互，极大地简化了文件操作的复杂性。例如，当应用程序调用 read 函数读取文件内容时，这个请求首先会通过系统调用接口进入内核空间，然后由 VFS 接收。VFS 根据文件的路径信息，确定该文件所属的具体文件系统，并将 read 请求转发给对应的文件系统驱动程序。文件系统驱动程序根据 VFS 传递的参数，在磁盘或其他存储设备上读取相应的数据，并将数据返回给 VFS。VFS 再将数据返回给应用程序，完成整个文件读取操作。

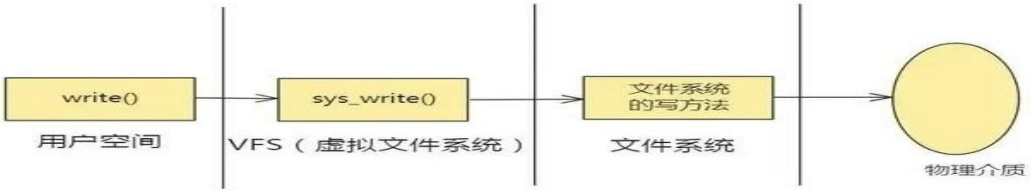
比如 Linux 写一个文件：int ret = write(fd, buf, len);

调用了 write()系统调用，它的过程简要如下：

首先，勾起 VFS 通用系统调用 sys_write()处理。

接着，sys_write()根据 fd 找到所在的文件系统提供的写操作函数，比如 op_write()。

最后，调用 op_write()。



在 Linux 系统中，常见的文件系统操作，如创建文件、读取文件、写入文件、删除文件、打开文件、关闭文件、重命名文件、获取文件属性等，VFS 都为它们提供了统一的系统调用接口。例如，当我们使用 open 函数来打开一个文件时，无论这个文件是存储在本地的 ext4 文件系统上，还是位于远程的 NFS 文件系统中，我们调用 open 函数的方式和参数都是一样的。同样，read 函数用于读取文件内容，write 函数用于写入文件内容，close 函数用于关闭文件，这些函数的使用方式不会因为底层文件系统的不同而改变。对于应用程序开发者来说，他们只需要熟悉这些统一的接口，就可以轻松地编写与文件系统交互的代码，而无需花费大量时间去了解不同文件系统的复杂细节。

两个数据结构：每种注册到内核的文件系统类型以 struct file_system_type 结构表示，每种文件系统类型中都有一个链表，指向所有属于该类型的文件系统的超级块。当一个文件系统挂载到内核文件系统的目录树上，会生成一个挂载点，用来管理所挂载的文件系统的信息。

该挂载点用一个 struct vfsmount 结构表示。

(2) 主要数据结构

Linux 文件系统基于一系列核心概念和对象构建，其中包括超级块 Superblock、索引节点 Inode、目录项 Dentry 和文件对象 File Object。VFS 通过这些抽象对象模型来管理文件系统。

Superblock 是文件系统的全局元数据，会有多个备份；Superblock, Inode, 和 Dentry 这些元数据存储在磁盘上，当访问文件时，它们会被缓存到内存中；File Object 在访问文件时产生，只存在于内存中。

超级块 Superblock

超级块本质上是文件系统的元数据，它存储了整个文件系统的关键控制信息，包括文件系统的类型、块大小、空闲和已用的块和 inode 的数量等等（即元数据的元数据）。每个挂载的文件系统都有一个 Superblock，它在文件系统首次挂载时从存储设备读入内存，它为文件系统的管理和操作提供了全局的视角，比如通过超级块可以快速了解文件系统的基本属性，以及获取文件系统中其他重要数据结构的位置信息。

```
struct super_block {
/*  全局链表元素  */
struct list_head s_list;
/*  底层文件系统所在的设备  */
dev_t s_dev;
/*  文件系统中每一块的长度  */
unsigned long s_blocksize;
/*  文件系统中每一块的长度（以 2 为底的对数）  */
unsigned char s_blocksize_bits;
/*  是否需要向磁盘回写  */
unsigned char s_dirt;
unsigned long long s_maxbytes; /* Max file size */
/*  文件系统类型  */
struct file_system_type *s_type;
/*  超级块操作方法  */
const struct super_operations *s_op;
struct dquot_operations *dq_op;
struct quotactl_ops *s_qcop;
const struct export_operations *s_export_op;
unsigned long s_flags;
unsigned long s_magic;
/*  全局根目录的 dentry */
```

```
struct dentry *s_root;
struct rw_semaphore s_umount;
struct mutex s_lock;
int s_count;
int s_need_sync;
atomic_t s_active;
#ifdef CONFIG_SECURITY
void *s_security;
#endif
struct xattr_handler **s_xattr;
/*  超级块管理的所有 inode 的链表  */
struct list_head s_inodes; /* all inodes */
/*  脏的 inode 的链表  */
struct list_head s_dirty; /* dirty inodes */
struct list_head s_io; /* parked for writeback */
struct list_head s_more_io; /* parked for more writeback */
struct hlist_head s_anon; /* anonymous dentries for (nfs) exporting */
/* file 结构的链表，该超级块上所有打开的文件  */
struct list_head s_files;
/* s_dentry_lru and s_nr_dentry_unused are protected by dcache_lock */
/*  不再使用的 dentry 的 LRU 链表  */
struct list_head s_dentry_lru; /* unused dentry lru */
int s_nr_dentry_unused; /* # of dentry on lru */struct block_device *s_bdev;
struct mtd_info *s_mtd;
/*  相同文件系统类型的超级块链表的节点  */
struct list_head s_instances;
struct quota_info s_dquot; /* Diskquota specific options */int s_frozen;
wait_queue_head_t s_wait_unfrozen;char s_id[32]; /* Informational name */void *s_fs_info;
/* Filesystem private info */
fmode_t s_mode;/*
* The next field is for VFS *only*. No filesystems have any business
* even looking at it. You had been warned.
*/
struct mutex s_vfs_rename_mutex; /* Kludge *//* Granularity of c/m/atime in ns.
Cannot be worse than a second */
u32 s_time_gran;/*
* Filesystem subtype. If non-empty the filesystem type field
* in /proc/mounts will be "type.subtype"
*/
char *s_subtype;/*
* Saved mount options for lazy filesystems using
* generic_show_options()
*/
char *s_options;
```

};

s_list：是一个 list_head 结构体对象。list_head 结构体如下定义。内核中使用一个双向环形链表将所有超级块连起来管理，即每个超级块的 s_list 属性都包含了指向内核链表中前一个元素和后一个元素的指针。全局变量 super_blocks 用于指向链表中第一个元素。Linux 内核经常用该对象间接定义双向循环链表来管理数据。

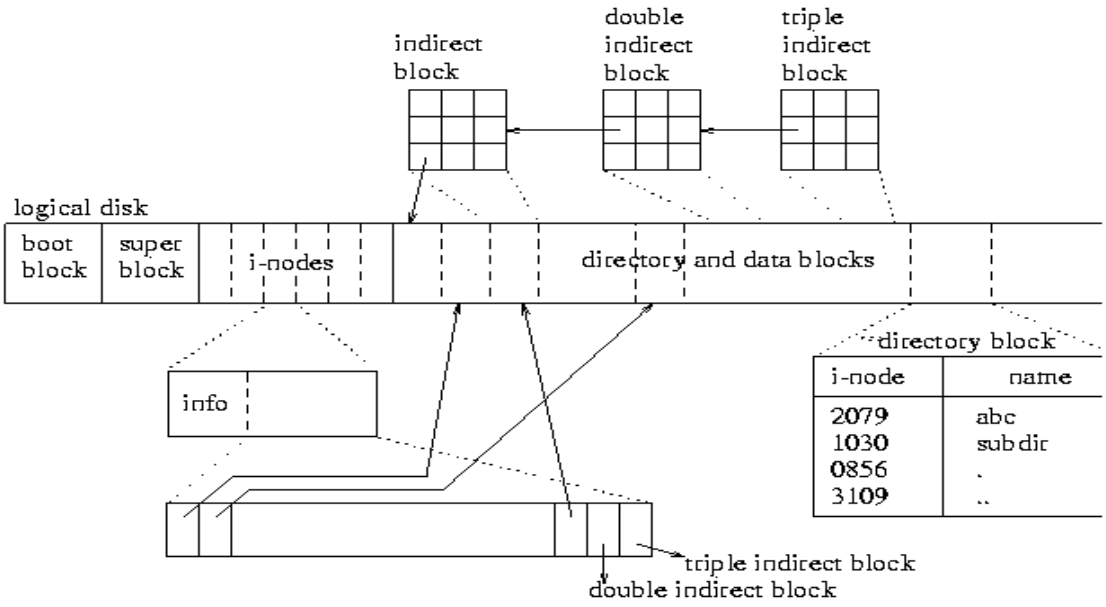
```
struct{
    list_head *prev;
    list_head *next;
}
```

s_blocksize：文件系统中数据块大小，单位是字 s_dirt：脏位，在具体的硬件设备中有关于其文件系统的数据，将设备挂载以后，会用其关于文件系统的数据来初始化内存中的 super_block 结构体对象。而 VFS 是允许对超级块对象进行修改的，修改后的数据最终是要写回磁盘对应区域的。s_dirt 用于判断超级块对象中数据是否脏了即被修改过了，即与磁盘上的超级块区域是否一致。s_dirty：脏 inode 的双向循环链表，用于同步内存数据和底层存储介质。当我们在用户去用 open 打开一个文件，内存中会创建 dentry 和 inode，当我们用 write 往文件中写入数据，则该 inode 脏了，将其加入到 s_dirty 链表 s_files。s_type：是指向 file_system_type 类型的指针，file_system_type 结构体用于保存具体的文件系统的信息。s_op：super_operations 结构体类型的指针，因为一个超级块对应一种文件系统，而每种文件系统的操作函数可能是不同的。super_operations 结构体由一些函数指针组成，这些函数指针用特定文件系统的超级块区域操作函数来初始化。比如里边会有函数实现获取和返回底层文件系统 inode 的方法。s_inodes：是一个 list_head 结构体对象，指向超级块对应文件系统中的所有 inode 索引节点的链表。

超级块对文件系统至关重要，因此每个文件系统都会存储多个冗余副本，通常存储在文件系统的多个不同位置，以防主超级块发生损坏。这些备份分布在整个文件系统的不同块组（block groups）中，确保在主超级块损坏时可以从备份中恢复。

查看超级块备份信息：`dumpe2fs /dev/foo | grep -i superblock`

查看 Superblock 的命令：对于不同类型的文件系统，可能会有所不同。ext 系列（Ext2, Ext3, Ext4）使用 `dumpe2fs` 命令查看 superblock 信息。



索引节点 Inode

在 Linux 文件系统中，每个文件和目录都有一个唯一的 Inode，它包含了关于文件的所有元数据（不包括文件名，文件名是通过目录项来管理的），如文件的大小、权限、所有者、时间戳和数据块位置，数据块的位置包括直接地址、一级索引、二级索引和三级索引。

如何查看 inode 相关信息：

使用 ls 命令。ls 命令是最常用的命令之一，用于列出目录内容。可以使用 -i 选项来显示文件或目录的 inode 号。

使用 stat 命令。stat 命令提供了关于文件或文件系统状态的详细信息，包括 inode 号在内的详细信息，如文件大小、块数、IO 块大小、文件类型、所有者、权限等。

使用 df -i 命令。使用 df -i 命令可以帮助用户了解每个挂载的文件系统中有多少 inode、已经使用了多少，以及还剩余多少。

目录项 Dentry

Dentry 对象用于表示文件系统中的目录和文件，是文件名和 Inode 之间的桥梁。用于将文件名（用户使用的文件标识）与其对应的 inode（文件系统中的文件标识）关联起来。目录项中包含了文件名、指向 inode 的指针以及一些其他的元数据。在文件系统的路径中，每一部分都对应一个目录项，例如/home/user/data.txt 这个路径中，/、home、user 和 data.txt 分别是一个目录项。通过目录项，VFS 可以快速地从文件系统的根目录开始，沿着目录层次结构找到目标文件或目录的 inode，从而实现对文件的访问。

通过将文件名从 inode 中分离出来，文件系统可以更有效地管理文件名和路径查找。

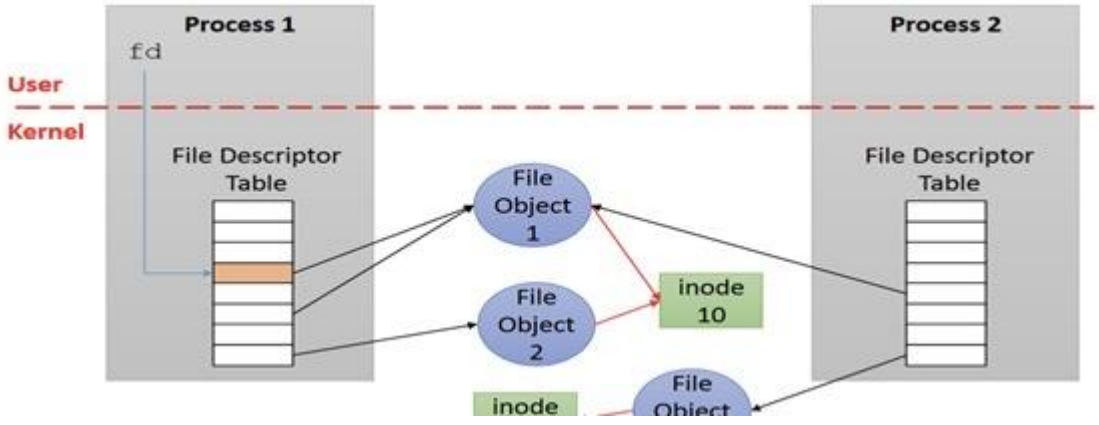
这样会使得文件的目录项变小，从而整个文件目录会变小，进行文件检索时会降低 I/O 时间开销。另外，可以更灵活地处理文件和目录的重命名、移动等操作。在重命名或移动文件时，只需修改 dentry 中的文件名或父目录指针，而无需修改 inode。

Dentry cache 即目录缓存（dentry cache），是一个用于加速文件系统访问的机制，它在内存中缓存了最近使用的 dentry 。当文件系统需要访问一个文件时，它首先查看目录缓存中是否存在该文件的 dentry。如果存在，文件系统可以直接使用这个缓存的 dentry，从而避免了在磁盘上查找文件名和对应 inode 的开销。这种缓存减少了磁盘 I/O 操作，可以加速对频繁访问文件的操作。

文件描述符(File Descriptors FD)和 File Object

文件描述符 FD 是一个非负整数，它在程序中作为一个简单的索引或标识符使用，用于访问打开的文件或其他数据流（如管道和网络连接）。每当一个进程打开一个文件（使用如 open 或 create 等系统调用）时，操作系统会返回一个文件描述符。这个描述符可以用于后续的文件操作，比如读取（read）、写入（write）、定位（lseek）或关闭（close）文件。

文件对象 (File Objects)是内核中的一个数据结构，是表示打开文件的 “活动记录”，当一个文件被打开时，VFS 会创建一个对应的文件对象，它包含了打开文件的所有信息，即指向 inode 的指针、当前文件的读写位置、文件操作的状态标志以及指向文件操作函数集的指针等信息。文件对象主要用于在文件打开期间，记录和管理文件的操作状态，例如当我们对一个打开的文件进行读写操作时，文件对象会记录当前的读写位置，以便下次继续进行读写操作。文件对象还提供了对文件进行各种操作的接口，这些接口最终会调用 inode 中定义的文件操作函数集来完成实际的文件操作。



每个文件描述符都对应一个文件对象。当一个文件被打开时，操作系统创建一个新的文件对象，并将一个文件描述符映射到这个文件对象上。文件描述符实际上是文件对象在用户程序中的代表或引用。

多个文件描述符可以指向同一个文件对象。通常发生在一个文件被多次打开，或者当文件描述符被复制（如通过 dup 或 fork 系统调用）时。这种机制允许多个进程或同一进程的不同部分独立地操作同一个文件。

文件描述符通常由具体的进程局部管理，每个进程都有自己的文件描述符表 FDT(File Descriptors Table)。相反，文件对象由操作系统全局管理，因为它们涉及到文件系统的具体实现和磁盘上的数据。

(3) 文件操作流程

以应用程序中调用 open 函数打开一个文件为例：

应用程序发起调用：应用程序中执行 open 函数，例如 int fd = open("/home/user/data.txt", O_RDONLY);，这里传递了文件的路径/home/user/data.txt 以及打开模式 O_RDONLY（表示只读打开）。此时，应用程序通过系统调用接口陷入内核态，将控制权交给内核中的 VFS 模块来处理这个请求。

路径解析与目录项查找：VFS 首先会对传入的文件路径进行解析。它从根目录/开始，逐步解析路径中的每一个部分。在这个过程中，VFS 会利用目录项高速缓存（dentry cache）来加速查找过程。目录项高速缓存中存储了最近访问过的目录项信息，当 VFS 解析路径时，会首先在缓存中查找对应的目录项。如果缓存命中，就可以直接获取该目录项的相关信息，避免了对磁盘的直接访问，大大提高了查找效率；如果缓存没有命中，VFS 就需要从磁盘上读取目录项信息，并将其添加到缓存中，以便下次访问时能够快速命中。对于/home/user/data.txt 这个路径，VFS 会依次查找根目录/的目录项、home 目录的目录项、user 目录的目录项，最后找到 data.txt 文件的目录项。每个目录项都包含了文件名以及指向对应索引节点（inode）的指针。

索引节点获取与文件对象创建：找到了文件目录项，VFS 可以获取到对应的索引节点。索引节点中存储了文件的元数据，如文件的大小、权限、所有者、创建时间、修改时间、数据块位置信息等。如果索引节点不在内存中，VFS 会从磁盘上读取该索引节点并将其加载到内存中。接下来，VFS 会为打开的文件创建一个文件对象。文件对象中包含了指向索引节点的指针、当前文件的读写位置、文件操作的状态标志以及指向文件操作函数集的指针等信息。文件对象的创建标志着文件已经被成功打开，应用程序可以通过返回的文件描述符

（ fd ）来对文件进行后续的操作。

常见文件操作

文件打开：当用户请求打开一个文件时，文件系统首先通过路径名解析过程，利用 Dentry 对象快速定位到文件的 Inode。然后，系统根据 Inode 中存储的信息，创建一个 File Object，这个对象将用于后续的读写操作。

文件写入：文件的写入操作首先需要定位到文件的 Inode，然后根据 Inode 中的数据块指针找到数据应该写入的位置。如果需要的话，文件系统可能还会分配新的数据块。数据写入完成后，相关的 Inode 信息（如文件大小和修改时间）会被更新。

文件读取：文件读取操作也是从定位 Inode 开始的。系统通过 Inode 中的数据块指针读取数据块，然后将数据传输到请求读取的进程中。File Object 中的文件指针随着读取的进行而更新。

2) 类 EXT2 的文件系统设计

请参阅 Linux 的 EXT2 文件系统与类 EXT2 的设计.ppt 文件。

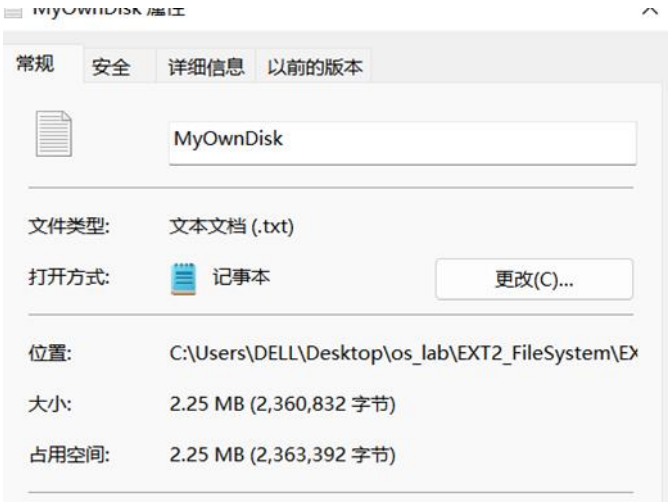
5. 实验步骤

1) 理解 Ext2 文件系统的设计思想, 自顶向下进行设计。

阅读 EXT2 文件系统数据结构、磁盘布局、底层操作等，构建程序基本框架。

2) 创建大小为 2.25MB 的 txt 文件，模拟磁盘。

可以用命令 `fsutil file createnew [filename].txt [size]` 来创建指定大小的文件，我们填充 `filename` 以及 `size(2360832byte)`，创建大小为 2.25MB 的 txt 文件：



模拟磁盘意味着我们要一次性以二进制读写 512byte 的数据，那我们如何实现以二进制数据读写，而不是将读取的字节转化为二进制？文件有多种使用标识：

'r':默认值，表示从文件读取数据。
'w':表示要向文件写入数据，并截断以前的内容
'a':表示要向文件写入数据，添加到当前内容尾部
'r+':表示对文件进行可读写操作（删除以前的所有数据）
'r+a'：表示对文件可进行读写操作（添加到当前文件尾部）
'b':表示要读写二进制数据

我们将 fopen()函数的 mode 字段设置成”rb+”，就可以实现以二进制的方式读写数据。另外，模拟磁盘意味着我们要找到文件中指定的位置进行读写，那我们如何将读写指针移动到特定的位置进行读写操作？可以用函数：fseek (FILE *fp, long offset, int origin);设置 offset 这一项，就可以将读写指针移动到距离文件起始位置 offset 字节处(origin=0)。

3) 自底向上编写代码, 实现文件系统。

(1) 编写程序，定义文件系统组描述符、目录项、inode 结点等数据结构。定义常驻内存变量。

常驻内存变量主要包括组描述符、打开文件表、当前目录、当前路径、上次分配的索引节点号、上次分配的数据块号等，具体取决于自己的设计。

(2) 编写程序，定义文件系统底层操作。

包括加载和更新组描述符、索引节点、数据块、目录项、数据块位图、索引节点位图等，分配数据块和索引节点，释放数据块和索引节点，索引节点初始化、加载和更新一级索引、二级索引等。

(3) 编写程序，定义文件系统命令层接口函数。

这里实现文件系统提供的操作命令，主要包括创建文件、删除文件、读文件、写文件、打开文件、关闭文件、列目录、建立目录、删除目录、更改目录、修改文件访问权限等，以及 login、password、format 等，具体取决于自己的设计。

(4) 编写程序，实现系统初始化以及主界面。

包括磁盘格式化、内存初始化，具体完成清空文件打开表、清空磁盘(文件中全部写入‘\0’)、初始化组描述符、创建根目录等。文件系统主界面可以是菜单形式，列出系统的全部命令。

4) 测试数据设计

设计测试数据文件，测试命令尽可能覆盖系统的所有功能。

6. 验收要求

验收包括：README 、程序源码、程序运行及回答问题

基本要求：

1) 根据实验内容要求，实现一个类 Ext2 的多级文件系统；

2) 至少实现下列几条命令：login,ls,create,delete,cd,close,read,write,format,password

3) 列目录时要列出文件类型、文件名、创建时间、修改时间、最后访问时间和修改时间；

4) 源文件可以进行读写保护。

回答问题：

设计文件系统的数据结构时遇到的挑战问题；

实现底层函数时遇到的挑战问题；

调试代码时遇到的挑战问题。